

Lab1:Data lab 实验报告

彭磊 25800190026

一、终端运行./check_ops.py bits.c 得到的结果

```
ladinp@LAPTOP-OIV2DLCS:~/lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 7/16 operators
roundEvenPow2: 21/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 30/52 operators
floatScaleThreeHalves: 34/60 operators
floatRoundEven: 27/65 operators
float_i2f: 26/40 operators
bitCount: 26/40 operators
bitReverse: 32/34 operators
All 19 functions passed operator checks.
ladinp@LAPTOP-OIV2DLCS:~/lab1$
```

二、终端运行./btest 得到的结果

```
ladinp@LAPTOP-OIV2DLCS:~/lab1$ ./btest
Score  Rating  Errors  Function
1      1       0      signMask
2      2       0      bitXor
3      3       0      negativePart
4      4       0      copyByteWithin
4      4       0      logicalShift
4      4       0      swapNibblePairs
4      4       0      secondLowestZeroBit
5      5       0      oddParity
5      5       0      rotateRightBits
5      5       0      roundEvenPow2
5      5       0      midpointTowardFirst
7      7       0      isBetweenEitherOrder
7      7       0      mul5Sat
7      7       0      classifyAdd3
7      7       0      floatScaleThreeHalves
10     10      0      floatRoundEven
10     10      0      float_i2f
10     10      0      bitCount
10     10      0      bitReverse
Total points: 110/110
```

三、函数实现思路描述

P1: signMask

将整数 1 左移 31 位，使最高位为 1、其余位为 0，得到符号位掩码 0x80000000。

P2: bitXor

利用德摩根定律，将异或运算改写为仅使用取反 $\sim$ 和与 $\&$ 的表达式。 x 与 y 的取反运算后进行与运算得到的 1 的位置为 x 为 1 的位置的二者不同位置， y 与 x 的取反同理得到 x 为 0 的位置而与 y 不同的位置，二者取与运算，只有在 x 和 y 相同的位置 1，然后取反得到异或

P3: negativePart

利用补码性质，通过 $\sim x + 1$ 计算 $-x$ 。将 x 算术右移 31 位，得到符号掩码：负数对应全 1，非负数对应全 0。 $-x$ 与 mask 取与运算，若非负则 neg 为负，mask 为 0，返回 0，否则 neg 为正，mask 为 0xFFFF，取与得到 neg 本身

P4: copyByteWithin

x 左移 $src * 8$ 位，然后与 0xFF 进行与运算，得到 src 位置的字节，再左移 $dst * 8$ 位，与 $x \sim (0xFF \ll (dst \ll 3))$ 进行或运算，得到结果

P5: logicalShift

$\sim((1 \ll 31) \gg n) \ll 1$ 为前 n 位都是 0，后面都是 1，与 $x \gg n$ 进行与运算使得前 n 位全为零，后面不变，实现逻辑右移

P6: swapNibblePairs

通过移位和或运算构造 0x0F0F0F0F 掩码，提取每个字节的低四位和高四位。将低四位左移四位、高四位右移四位，再通过或运算合并，实现每个字节内部高低四位的交换。

P7: secondLowestZeroBit

先对 x 取反，将原来的零位转换为 1。利用 $y \& (y - 1)$ 清除最低位的 1，再利用 $y \& (\sim y + 1)$ 提取剩余结果中最低位的 1，从而定位原数中第二个最低位的零。若不足两个零位，则返回零。

P8: oddParity

利用异或运算可以反映参与运算的二进制位中 1 的数量的奇偶性。依次将 x 右移 16、8、4、2、1 位，并与原值异或，将各位信息逐步折叠到最低位。最后根据最低位的值判断原数中 1 的个数是奇数还是偶数。

P9: rotateRightBits

先将移位量与 31 相与，使其限制在 0 到 31 之间。再分别计算右移和左移的结果，并利用掩码去除算术右移产生的符号扩展位。最后将两部分通过或运算合并，实现循环右移。

P10: roundEvenPow2

将 x 右移 n 位得到商，通过掩码提取低 n 位得到余数。将余数与半个间隔比较：小于中点时向下舍入，大于中点时向上舍入；恰好位于中点时，根据商的奇偶性决定是否进位。最后将调整后的商左

移 n 位，得到最近的 2^n 的整数倍。

P11: midpointTowardFirst

利用 $\lfloor (x+y)/2 \rfloor = (x \& y) + ((x \oplus y) \gg 1)$ 计算中值，避免直接计算 $x+y$ 造成溢出。然后判断第一个参数 x 是否大于 y ，并结合两数的奇偶性决定是否加一，使结果在中点为半整数时更接近 x 。

P12: isBetweenEitherOrder

分别通过补码的加法构造 $x-a$ 、 $b-x$ 、 $x-b$ 和 $a-x$ ，判断 x 是否位于两个端点构成的闭区间内。由于减法可能发生有符号溢出，需要结合操作数和结果的符号位检测溢出，并修正大小判断。最后合并两种端点顺序下的区间判断结果。

P13: mul5Sat

利用 $5x=4x+x$ ，通过左移两位和加法计算乘积。分别检查计算 $4x$ 和计算 $4x+x$ 时是否发生有符号溢出。若发生溢出，则根据输入的符号选择输出的极值；否则返回正常乘积。

P14: classifyAdd3

先计算两次加法的结果，再利用操作数和结果的符号位判断正溢出与负溢出。根据溢出方向返回 1、-1 或 0，分别表示精确数学和超过最大值、低于最小值或处于合法范围内。

P15: floatScaleThreeHalves

将输入提取符号位、指数和尾数。根据指数判断输入属于零或非规格化数、规格化数，还是无穷大或 NaN。对有限数值计算 1.5 倍，并通过舍入位和保留结果的最低位实现最近偶数舍入。最后调整指数，处理指数溢出，并重新组合浮点数的各个字段。

P16: floatRoundEven

提取浮点数的符号位、指数和尾数，根据指数判断数值范围。对于绝对值小于 0.5 的数，返回带原符号的零；对于其他有限数，利用移位，分离整数部分和小数部分，并根据最近偶数规则决定是否进位。最后确定舍入后整数的最高有效位，重新构造浮点数表示。对于 NaN 和无穷大，则直接返回原值。

P17: float_i2f

先处理零和负数的情况，确定符号位并取得整数的绝对值。通过循环寻找最高有效位的位置，以确定浮点数的指数。当有效位数不超过单精度尾数精度时，直接左移对齐；否则右移并检查被舍弃的位，根据最近偶数规则进行舍入。若舍入产生额外进位，则调整指数，最后组合符号位、指数和尾数。

P18: bitCount

通过构造合适的掩码，实现逐级统计各部分二进制位中 1 的数量。先统计每两位中的 1，再合并为四位、八位、十六位，最终合并整个 32 位整数的计数结果。

P19: bitReverse

通过分组交换实现二进制位反转。先交换相邻位，再交换相邻的两位组、四位组、字节以及高低 16 位。每一步都将交换范围扩大一倍，最终使原整数的二进制位顺序完全反转。

